# Proyectos de Programación 26/27
# Práctica de pruebas – 16.11.2026

En esta práctica vas a aprender a **escribir pruebas con GoogleTest** como las que pide la Entrega 2: qué es una prueba unitaria, de caja negra, de caja blanca, de integración y de rendimiento, cómo se escribe cada una y, sobre todo, **para qué sirve una prueba: para encontrar errores antes que el usuario**.

Trabajaréis **por parejas**, con dos repositorios públicos que ya están preparados (no tienes que traer nada):

| Repositorio | Qué es | Qué haces con él |
|---|---|---|
| **Referencia** `pp-lab3-tests-referencia` | Un **hotel** (Habitacion, Suite, Hotel, guardar/cargar) con **24 pruebas** ya hechas y bien hechas | Lo descargas, lo compilas y lo **lees**. No se modifica: es el modelo del que copiar |
| **Ejercicio** `pp-lab3-tests-ejercicio` | Una **biblioteca** (Libro, Novedad, Biblioteca, guardar/cargar) con **la misma estructura**, pero con **un solo test** | Creáis vuestra copia y escribís las pruebas **copiando y adaptando** las del hotel |

**Atención**

**El código de la biblioteca tiene 4 errores escondidos.** No los busquéis leyendo el código: si vuestras pruebas están bien hechas, **ellas los encontrarán** (se pondrán en rojo). La regla de la práctica es: **primero el test en rojo; después, el arreglo**.

**Cómo leer esta guía**

- Los recuadros negros son la **terminal**. Lo que tienes que escribir está en `amarillo`; lo que va delante (`PS C:\Users\alumno>` o `alumno@MacBook … %`) es el *prompt*: **no lo escribas**. Lo que aparece en gris debajo es lo que responde el ordenador. Los `…` son líneas que se han omitido.
- Cuando Windows y Mac son distintos verás dos recuadros: **usa solo el de tu sistema**.
- Los recuadros verdes dicen lo que deberías ver; los rojos, qué hacer si ves otra cosa.
- **Por parejas:** trabajad en **un solo ordenador** y **cambiad de quién escribe en cada paso**. El otro lee la guía y revisa.

| Parte | Qué haces | Tiempo |
|---|---|---|
| 1 | Comprobar las herramientas y descargar la referencia | 10 min |
| 2 | Recorrer las pruebas de la referencia y ver una en rojo a propósito | 10 min |
| 3 | Crear vuestro repositorio de ejercicio | 10 min |
| 4-7 | Escribir las pruebas de la biblioteca y encontrar los 4 errores | 60-70 min |
| 8 | (Opcional) Prueba de rendimiento | — |
| Para casa | Lo mismo en el proyecto de vuestro grupo | — |


Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

# 1. Preparar el equipo y descargar la referencia (10 min)

## 1.1 Comprobar que tienes las herramientas

Son las mismas que en la práctica de CMake. Cada comando debe responder con un número de versión, **no con un error**:

Windows · PowerShell

```
PS C:\Users\alumno\PP> g++ --version
g++.exe (Rev2, Built by MSYS2 project) 14.2.0
PS C:\Users\alumno\PP> cmake --version
cmake version 3.31.0
PS C:\Users\alumno\PP> git --version
git version 2.51.0.windows.2
PS C:\Users\alumno\PP> ninja --version
1.12.1
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % g++ --version
Apple clang version 17.0.0 (clang-1700.0.13.3)
alumno@MacBook-de-alumno PP % cmake --version
cmake version 4.1.2
alumno@MacBook-de-alumno PP % git --version
git version 2.39.5 (Apple Git-154)
```

Si no tienes la carpeta `PP`, créala en tu carpeta personal con `mkdir PP` y entra con `cd PP`. Los números de versión pueden ser distintos. **En Mac no hace falta `ninja`.**

> **Atención**
>
> **¿Te falta alguna herramienta?** Tienes el **Anexo · Instalar las herramientas** al final de esta guía. Instalar lleva unos 15-20 minutos: mientras tanto, trabajad en el ordenador de tu pareja.
>
> **Windows, `ninja` no se reconoce:** en la terminal **MSYS2 MSYS**, `pacman -S --needed mingw-w64-x86_64-ninja`, y después cierra y abre PowerShell.

## 1.2 Descargar, compilar y probar la referencia

Desde tu carpeta `PP`:

Windows · PowerShell

```
PS C:\Users\alumno\PP> git clone https://github.com/jcor-cunpir/pp-lab3-tests-referencia.git
Cloning into 'pp-lab3-tests-referencia'...
…
PS C:\Users\alumno\PP> cd pp-lab3-tests-referencia
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cmake -S . -B build -G Ninja
…
-- Build files have been written to: C:\Users\alumno\PP\pp-lab3-tests-referencia\build
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cmake --build build
…
[…] Linking CXX executable tests_gestor.exe
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % git clone https://github.com/jcor-cunpir/pp-lab3-tests-referencia.git
Cloning into 'pp-lab3-tests-referencia'...
…
alumno@MacBook-de-alumno PP % cd pp-lab3-tests-referencia
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cmake -S . -B build
…
-- Build files have been written to: /Users/alumno/PP/pp-lab3-tests-referencia/build
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cmake --build build
…
[100%] Built target tests_gestor
```

La primera vez, `cmake -S . -B build` **descarga GoogleTest** de internet: tarda un poco más. Ahora, los tests:


Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cd build
PS C:\Users\alumno\PP\pp-lab3-tests-referencia\build> ctest --output-on-failure
    Start 1: GestorTests
1/1 Test #1: GestorTests .....................   Passed    0.40 sec

100% tests passed, 0 tests failed out of 1
PS C:\Users\alumno\PP\pp-lab3-tests-referencia\build> cd ..
```

Mac · Terminal

```
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cd build
alumno@MacBook-de-alumno build % ctest --output-on-failure
    Start 1: GestorTests
1/1 Test #1: GestorTests .....................   Passed    0.40 sec

100% tests passed, 0 tests failed out of 1
alumno@MacBook-de-alumno build % cd ..
```

**¿Qué ha pasado?**

`ctest` dice «1 test» porque para CMake hay **un solo** programa de pruebas, `tests_gestor`, que dentro tiene **todos** los tests de GoogleTest. Para verlos uno a uno, se ejecuta directamente:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> .\build\tests_gestor.exe
[==========] Running 24 tests from 5 test suites.
…
[ RUN      ] SuiteTest.ConDescuentoJustoConSieteNoches
[       OK ] SuiteTest.ConDescuentoJustoConSieteNoches (0 ms)
…
[==========] 24 tests from 5 test suites ran. (12 ms total)
[  PASSED  ] 24 tests.
```

Mac · Terminal

```
alumno@MacBook-de-alumno pp-lab3-tests-referencia % ./build/tests_gestor
[==========] Running 24 tests from 5 test suites.
…
[ RUN      ] SuiteTest.ConDescuentoJustoConSieteNoches
[       OK ] SuiteTest.ConDescuentoJustoConSieteNoches (0 ms)
…
[==========] 24 tests from 5 test suites ran. (12 ms total)
[  PASSED  ] 24 tests.
```

**✔ Deberías ver**

`[ PASSED ] 24 tests.` al final. Abre la carpeta en VS Code con `code .` para la parte 2.

**✘ Si en vez de eso ves...**

`Failed to clone repository: 'https://github.com/google/googletest.git'`: CMake necesita **internet** y **Git** para descargar GoogleTest. Comprueba la conexión y `git --version`, borra la carpeta `build` y vuelve a configurar.

`does not appear to contain CMakeLists.txt`: no estás en la carpeta del repositorio. `pwd` y `cd`.

`Does not match the generator used previously`: borra la carpeta `build` y vuelve a configurar.


Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

## 2. Recorrer las pruebas de la referencia (10 min)

En VS Code, abre la carpeta `tests/`. Hay un fichero por clase, y cada uno es un **tipo de prueba** distinto (lo que se ha visto en clase):

| Fichero | Tipo de prueba | La idea |
|---|---|---|
| `test_habitacion.cpp` | **Unitaria, caja negra** | Se prueba **una clase sola**, mirando solo lo que **promete** su `.hpp` (sin leer el `.cpp`): casos normales y **valores límite** (el 0, el primer valor válido, el primero no válido) |
| `test_suite.cpp` | **Unitaria, caja blanca** | Se escribe **mirando el código**: si hay un `if`, un test por cada camino, y otro **justo en el límite** del `if` |
| `test_hotel.cpp` | **Unitaria con *fixture*** | `TEST_F`: todos los tests parten del **mismo hotel** preparado en `SetUp()`, sin repetirlo en cada test |
| `test_almacen.cpp` | **Integración** | Varias piezas **juntas**: se guarda un hotel en un fichero real, se carga y se comprueba que vuelve igual |
| `test_rendimiento.cpp` | **Rendimiento** | No mira *qué* devuelve, sino **cuánto tarda** con muchos datos |

### 2.1 La anatomía de un test

`tests/test_habitacion.cpp`

```
TEST(HabitacionTest, ReservarDosVecesDevuelveFalse) {
    Habitacion h(101, 80.0);          // 1. PREPARAR: crear los objetos
    h.reservar();                     // 2. ACTUAR: llamar a lo que se prueba
    EXPECT_FALSE(h.reservar());       // 3. COMPROBAR: ¿pasa lo que promete el .hpp?
    EXPECT_TRUE(h.estaOcupada());
}
```

`TEST(Grupo, Nombre)`: el grupo suele ser la clase (`HabitacionTest`) y el nombre dice **qué comprueba**, como una frase. **No hay `main()`**: lo pone GoogleTest (`GTest::gtest_main` en el `CMakeLists.txt`).

| Quiero comprobar... | Macro | Ejemplo en la referencia |
|---|---|---|
| Que dos valores son iguales (`int`, `string`, punteros...) | `EXPECT_EQ(real, esperado)` | `EXPECT_EQ(h.getNumero(), 101);` |
| Que dos `double` son iguales | `EXPECT_DOUBLE_EQ(real, esperado)` | `EXPECT_DOUBLE_EQ(s.precioEstancia(7), 1260.0);` |
| Que algo es verdadero / falso | `EXPECT_TRUE` / `EXPECT_FALSE` | `EXPECT_FALSE(h.estaOcupada());` |
| Que una llamada lanza una excepción de ese tipo | `EXPECT_THROW(llamada, tipo)` | `EXPECT_THROW(Habitacion(101, 0.0), std::invalid_argument);` |
| Que una llamada **no** lanza | `EXPECT_NO_THROW(llamada)` | `EXPECT_NO_THROW(Habitacion(101, 0.01));` |
| Lo mismo, pero **parando el test** si falla | `ASSERT_...` | `ASSERT_NE(h, nullptr);` antes de usar `h->` |

> **¿Qué ha pasado?**
>
> **Excepciones** (se explican el miércoles 18): cuando el constructor recibe datos que no valen, en lugar de crear un objeto incorrecto **«protesta»** con `throw std::invalid_argument(...)`. `EXPECT_THROW` comprueba que protesta cuando debe.

## 2.2 Ver un test en rojo, a propósito

Un test sirve cuando **falla si el código está mal**. Vamos a verlo: en `tests/test_habitacion.cpp`, línea 56, cambia `240.0` por `250.0`, **guarda** (`Ctrl+S` / `⌘S`), compila y pasa los tests:

`tests/test_habitacion.cpp, línea 56`

```
    EXPECT_DOUBLE_EQ(h.precioEstancia(3), 250.0);      // antes: 240.0
```

**Windows · PowerShell**

```
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> cmake --build build
PS C:\Users\alumno\PP\pp-lab3-tests-referencia> .\build\tests_gestor.exe
…
C:/Users/alumno/PP/pp-lab3-tests-referencia/tests/test_habitacion.cpp:56: Failure
Expected equality of these values:
  h.precioEstancia(3)
    Which is: 240
  250.0
    Which is: 250
[  FAILED  ] HabitacionTest.PrecioEstanciaMultiplicaPorLasNoches (0 ms)
…
 1 FAILED TEST
```

**Mac · Terminal**

```
alumno@MacBook-de-alumno pp-lab3-tests-referencia % cmake --build build
alumno@MacBook-de-alumno pp-lab3-tests-referencia % ./build/tests_gestor
…
/Users/alumno/PP/pp-lab3-tests-referencia/tests/test_habitacion.cpp:56: Failure
Expected equality of these values:
  h.precioEstancia(3)
    Which is: 240
  250.0
    Which is: 250
[  FAILED  ] HabitacionTest.PrecioEstanciaMultiplicaPorLasNoches (0 ms)
…
 1 FAILED TEST
```

> **¿Qué ha pasado?**
>
> Así se lee un rojo: **fichero y línea** del `EXPECT` que ha fallado (`:56`), **qué se ha obtenido** (`h.precioEstancia(3)` → *Which is: 240*) y **qué se esperaba** (`250.0`), y el **nombre del test**. Aquí el que está mal es el test (3 × 80 = 240). En la biblioteca será al revés: **el test estará bien y lo que estará mal será la clase**.

> **Atención**
>
> **Deja la línea 56 como estaba** (`240.0`), guarda y comprueba que vuelven a pasar los 24. La referencia no se modifica.

# 3. Crear vuestro repositorio de ejercicio (10 min)

## 3.1 Crearlo desde la plantilla (lo hace UNA persona de la pareja)

1. Entra en **https://github.com/jcor-cunpir/pp-lab3-tests-ejercicio** con tu cuenta de GitHub.
2. Botón verde **Use this template** → **Create a new repository**.
3. *Repository name*: `lab3-biblioteca`. Da igual **Public** o **Private**. Pulsa **Create repository**.
4. Opcional: **Settings → Collaborators → Add people** para añadir a tu pareja (así también puede subir cambios desde su ordenador).
5. Pestaña **Actions**: al cabo de uno o dos minutos, la primera ejecución de **CI** termina en ✔ **verde**.

## 3.2 Descargarlo, compilarlo y probarlo

Desde tu carpeta `PP` (en VS Code: **Terminal → Nueva terminal**, y `cd ..` para salir de la referencia). Cambia `tu-usuario` por tu usuario de GitHub:

Windows · PowerShell

```
PS C:\Users\alumno\PP> git clone https://github.com/tu-usuario/lab3-biblioteca.git
PS C:\Users\alumno\PP> cd lab3-biblioteca
PS C:\Users\alumno\PP\lab3-biblioteca> cmake -S . -B build -G Ninja
…
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
…
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
…
[  PASSED  ] 1 test.
PS C:\Users\alumno\PP\lab3-biblioteca> code .
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % git clone https://github.com/tu-usuario/lab3-biblioteca.git
alumno@MacBook-de-alumno PP % cd lab3-biblioteca
alumno@MacBook-de-alumno lab3-biblioteca % cmake -S . -B build
…
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
…
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
…
[  PASSED  ] 1 test.
alumno@MacBook-de-alumno lab3-biblioteca % code .
```

Se abre **otra ventana** de VS Code con la biblioteca. Trabajaréis con las dos a la vez: la **referencia** para copiar y la **biblioteca** para escribir. A partir de aquí, usa la terminal de la ventana de la biblioteca.

## 3.3 Las equivalencias

| Hotel (referencia) | Biblioteca (ejercicio) |
|---|---|
| `Habitacion(numero, precioNoche)` | `Libro(codigo, titulo)` |
| `reservar()` · `liberar()` · `estaOcupada()` | `prestar()` · `devolver()` · `estaPrestado()` |
| `precioEstancia(noches)` | `multa(dias)` |
| `Suite` (10 % de descuento desde 7 noches) | `Novedad` (multa más alta y recargo desde 7 días) |
| `Hotel`: `anadir`, `buscar`, `reservar`, `libres` | `Biblioteca`: `anadir`, `buscar`, `prestar`, `disponibles` |
| `guardarHotel` · `cargarHotel` · `hotel.txt` | `guardarBiblioteca` · `cargarBiblioteca` · `biblioteca.txt` |

> **Atención**
>
> **No todo es igual.** Antes de copiar un test, lee el **comentario del** `.hpp` del método en `include/`: dice exactamente qué promete. **Lo que dice ese comentario es lo que manda**: si un test y el comentario no coinciden, el que está mal es el test.


Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

## 4. Pruebas de Libro: unitarias de caja negra (20 min)

Abre `include/libro.hpp` y lee los comentarios. Después abre `tests/test_libro.cpp`: ya tiene un test de ejemplo. Tienes que escribir debajo de `>>> AQUÍ <<<` el equivalente de cada test de `test_habitacion.cpp` de la referencia:

| Test del hotel ( `test_habitacion.cpp` ) | Test de la biblioteca ( `test_libro.cpp` ) | Pista |
|---|---|---|
| `ConstructorGuardaLosDatos` | `ConstructorGuardaLosDatos` | Ya está hecho |
| `PrecioCeroLanzaExcepcion` | `CodigoCeroLanzaExcepcion` | Valor límite: código 0 |
| `PrecioMinimoPositivoNoLanza` | `CodigoUnoNoLanza` | El primer código válido |
| `NumeroNegativoLanzaExcepcion` | `TituloVacioLanzaExcepcion` | El título vacío es `""` |
| `ReservarCambiaElEstado` | `PrestarCambiaElEstado` | Resuelto abajo, como ejemplo |
| `ReservarDosVecesDevuelveFalse` | `PrestarDosVecesDevuelveFalse` | |
| `LiberarDejaLaHabitacionLibre` | `DevolverLoDejaDisponible` | |
| `PrecioEstanciaMultiplicaPorLasNoches` | `MultaMultiplicaPorLosDias` | 4 días → 2,0 EUR |
| `PrecioEstanciaConCeroNochesLanzaExcepcion` | `MultaConCeroDiasEsCero` y `MultaConDiasNegativosLanzaExcepcion` | **¡Distinto!** Lee el `.hpp`: ¿0 días vale? |

**Ejemplo resuelto**: copiar `ReservarCambiaElEstado` y adaptarlo. Se cambian la clase, el constructor, los métodos y el nombre del test:

```
// En la REFERENCIA (tests/test_habitacion.cpp):
TEST(HabitacionTest, ReservarCambiaElEstado) {
    Habitacion h(101, 80.0);
    EXPECT_TRUE(h.reservar());
    EXPECT_TRUE(h.estaOcupada());
}

// En la BIBLIOTECA (tests/test_libro.cpp), debajo de >>> AQUÍ <<<:
TEST(LibroTest, PrestarCambiaElEstado) {
    Libro l(1, "El Quijote");
    EXPECT_TRUE(l.prestar());
    EXPECT_TRUE(l.estaPrestado());
}
```

Escribe **dos o tres tests**, guarda, compila y pasa los tests. Repite hasta tenerlos todos:


Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

Windows · PowerShell

```
PS C:\Users\alumno\PP\lab3-biblioteca> cmake --build build
PS C:\Users\alumno\PP\lab3-biblioteca> .\build\tests_gestor.exe
…
[  PASSED  ] 4 tests.
```

Mac · Terminal

```
alumno@MacBook-de-alumno lab3-biblioteca % cmake --build build
alumno@MacBook-de-alumno lab3-biblioteca % ./build/tests_gestor
…
[  PASSED  ] 4 tests.
```

**✘ Si en vez de eso ves…**

**Error de compilación** en `test_libro.cpp` (`error: 'Habitacion' was not declared`, `has no member named 'reservar'`…): te has dejado algo del hotel sin cambiar. Ve a la línea que dice el error.
**Un test en rojo**: ¡puede que hayas encontrado un error! Sigue en 4.1.

## 4.1 Cuando un test sale en rojo

1. Lee el rojo como en el paso 2.2: ¿qué test, qué línea, qué se obtiene y qué se esperaba?
2. Lee el comentario del `.hpp`. **¿Está mal el test o está mal la clase?** Si el test no hace lo que dice el comentario, arregla el test.
3. Si el test está bien, **has encontrado un error de la biblioteca**. Antes de arreglarlo, súbelo así, **en rojo**:

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add tests/test_libro.cpp
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "test(libro): pruebas de caja negra de Libro"
PS C:\Users\alumno\PP\lab3-biblioteca> git push
```

En GitHub, pestaña **Actions**: la ejecución nueva termina con una **✘ roja**. Pulsa en ella → **build-and-test** → despliega el paso **Tests**: es el mismo rojo que en tu ordenador. **Así se ve en la defensa un proyecto con un error.**

4. Ahora **arregla la clase** en `src/libro.cpp` (suele ser una línea). Compila y pasa los tests hasta que estén todos en verde.
5. Sube el arreglo con un mensaje que diga qué arreglas:

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\lab3-biblioteca> git add src/libro.cpp
PS C:\Users\alumno\PP\lab3-biblioteca> git commit -m "fix(libro): <qué has arreglado>"
PS C:\Users\alumno\PP\lab3-biblioteca> git push
```

**✔ Deberías ver**

Todos los tests de Libro en verde en tu ordenador y, en **Actions**, una ✘ roja (el test) seguida de un ✔ verde (el arreglo). **Primer error encontrado.** Quedan 3.

**✘ Si en vez de eso ves…**

`git push` pide usuario y contraseña, o da `Permission denied / 403`: el repositorio es de tu pareja y no te ha añadido como colaborador (paso 3.1, punto 4), o tienes otra cuenta de GitHub abierta. Trabajad en el ordenador de quien creó el repositorio.
`Author identity unknown`: configura Git como en la práctica 1: `git config --global user.name "Tu Nombre"` y `git config --global user.email "tu@email"`.

## 5. Pruebas de Novedad: caja blanca y un fichero de tests nuevo (15 min)

Ahora el fichero de tests **no existe**: hay que crearlo y **decirle a CMake que lo compile**. Es lo mismo que tendréis que hacer en vuestro proyecto con cada clase nueva.

1. En VS Code, botón derecho sobre la carpeta `tests` → **Nuevo archivo** → `test_novedad.cpp`.
2. Copia dentro **todo** el contenido de `tests/test_suite.cpp` de la referencia.
3. Cambia el `#include "suite.hpp"` por `#include "novedad.hpp"`, `SuiteTest` por `NovedadTest` y adapta cada test (abajo).
4. Añádelo al `CMakeLists.txt`, en `add_executable(tests_gestor …)`:

CMakeLists.txt

```
# >>> AQUÍ <<<  Cada vez que creéis un fichero de tests nuevo en tests/, añadidlo aquí.
add_executable(tests_gestor
    tests/test_libro.cpp
    tests/test_novedad.cpp
    ${FUENTES}
)
```

**Caja blanca** significa escribir el test **mirando el código**. Abre `src/novedad.cpp`: hay un `if`. Igual que en `test_suite.cpp`, necesitas un test por cada camino del `if` y otro **justo en el límite** que dice el comentario del `.hpp`:

| Test de la Suite | Test de la Novedad | Valor esperado (calcúlalo con el comentario del `.hpp`) |
|---|---|---|
| `SinDescuentoConSeisNoches` | `SinRecargoConSeisDias` | 6 días → ¿? |
| `ConDescuentoJustoConSieteNoches` | `ConRecargoJustoConSieteDias` | 7 días → ¿? |
| `ConDescuentoConDiezNoches` | `ConRecargoConDiezDias` | 10 días → ¿? |
| `CeroNochesLanzaExcepcion` | `DiasNegativosLanzaExcepcion` | `multa(-1)` lanza |
| `PolimorfismoDesdeUnPunteroALaBase` | `PolimorfismoDesdeUnPunteroALaBase` | `std::unique_ptr<Libro>` que apunta a una `Novedad` |

**Truco**

Si después de añadir el fichero al `CMakeLists.txt` no aparecen los tests nuevos (`Running … tests` no cambia), o sale `undefined reference to main`, comprueba que has guardado el `CMakeLists.txt`. `cmake --build build` vuelve a configurar solo.

**✔ Deberías ver**

Si un test sale en rojo y el comentario del `.hpp` te da la razón: **segundo error**. Lo mismo que en el paso 4.1: commit del test (`git add tests/test_novedad.cpp CMakeLists.txt`) → push → rojo → arreglo en `src/novedad.cpp` → commit → push → verde.

## 6. Pruebas de Biblioteca: un fixture con TEST_F (20 min)

Crea `tests/test_biblioteca.cpp` copiando `tests/test_hotel.cpp` de la referencia, y añádelo al `CMakeLists.txt` igual que en el paso 5. El *fixture* es la clase que prepara los datos: cada `TEST_F` empieza con una biblioteca **nueva**:

tests/test_biblioteca.cpp (el principio)

```
#include "biblioteca.hpp"
#include "novedad.hpp"

class BibliotecaTest : public ::testing::Test {
protected:
    void SetUp() override {                      // se ejecuta ANTES de cada TEST_F
        biblioteca.anadir(std::make_unique<Libro>(1, "El Quijote"));
        biblioteca.anadir(std::make_unique<Libro>(2, "La Regenta"));
        biblioteca.anadir(std::make_unique<Novedad>(3, "Dune"));
    }
    Biblioteca biblioteca;
};

TEST_F(BibliotecaTest, TotalCuentaTodosLosLibros) {
    EXPECT_EQ(biblioteca.total(), 3u);
    EXPECT_EQ(biblioteca.disponibles(), 3);
}
```

Después, adapta los otros cinco tests de `test_hotel.cpp`: buscar uno que existe (con `ASSERT_NE(l, nullptr)` antes de usar `l->`), buscar uno que no existe, añadir un código repetido, prestar y prestar uno que no existe.

**¿Qué ha pasado?**

`3u`: `total()` devuelve un `size_t` (un entero **sin signo**), y la `u` dice que el 3 también lo es.
**¿Por qué `ASSERT` y no `EXPECT`?** Si `buscar` devuelve `nullptr` y el test sigue, `l->getTitulo()` rompe el programa de tests entero. `ASSERT` para el test en ese punto.

**✔ Deberías ver**

Si alguno sale en rojo y el comentario del `.hpp` te da la razón: **tercer error**. Test en rojo → push → arreglo → push.

## 7. Prueba de integración: guardar y cargar (15 min)

Crea `tests/test_almacen.cpp` copiando el de la referencia y añádelo al `CMakeLists.txt`. Es una prueba de **integración**: no prueba una clase sola, sino que `Biblioteca`, `Libro`, `Novedad` y el fichero funcionan **juntos**.

- En `GuardarYCargarDevuelveLoMismo`: una biblioteca con un `Libro` y una `Novedad` prestada, se guarda en `test_biblioteca.txt`, se carga y se comprueba **todo**: cuántos libros hay, el título, si está prestado y **el tipo** (`tipo()`).
- Los otros dos tests (fichero que no existe y línea mal escrita) se adaptan casi sin cambios. En el fichero roto, escribe una línea con el formato de la biblioteca: `Libro;abc;La Regenta;0`.
- Al final de cada test, `std::remove("…")` borra el fichero de prueba.

> **¿Qué ha pasado?**
> Los ficheros de prueba aparecen en la carpeta desde la que se ejecutan los tests. Si ejecutas `.\build\tests_gestor.exe` desde la raíz, en la raíz; con `ctest`, en `build/`. Por eso cada test borra lo que crea.

> **✔ Deberías ver**
> **Cuarto error** encontrado y arreglado: **unos 24 tests en verde** en tu ordenador y ✔ verde en **Actions**. En *Actions* se ve la historia de la práctica: cuatro rojos, cada uno seguido de su arreglo.

> **Truco**
> Este error lo podrías haber visto también **usando el programa**: ejecuta `gestor` dos veces seguidas y fíjate en el tipo de «Dune» antes y después de cargar. Las pruebas de integración encuentran lo que las unitarias no ven: cada pieza funciona sola, pero no juntas.

## 8. (Opcional) Prueba de rendimiento

Copia `tests/test_rendimiento.cpp` de la referencia como `tests/test_rendimiento.cpp` de la biblioteca: llena una biblioteca con 2000 libros y mide cuánto tardan 1000 búsquedas con `std::chrono`. El límite (`EXPECT_LT(ms, 1000)`) es muy generoso a propósito: el CI se ejecuta en máquinas compartidas, y la prueba sirve para detectar que algo se ha vuelto **mucho** más lento, no para medir con precisión.

## Para casa · Lo mismo, en el proyecto de vuestro grupo

> **Atención**
>
> **Viernes 20 de noviembre, 23:59:** se congela el repositorio de la Entrega 2. Tiene que tener **al menos 5 tests** que pasen y el CI de **Actions en verde**. En la defensa (lunes 23) os puedo preguntar por cualquier test: qué comprueba y por qué fallaría si la clase estuviera mal.

1. Por cada clase de vuestro proyecto, un fichero `tests/test_<clase>.cpp`, añadido a `add_executable(tests_gestor …)`.
2. Copiad los tests de la referencia que más se parezcan a vuestra clase y adaptadlos, como hoy. La tabla de abajo dice cuáles, como mínimo.
3. Antes de cada `git push`: `cmake --build build` y `ctest --output-on-failure` en vuestro ordenador.
4. Cuando hayáis terminado con la clase de ejemplo de la plantilla, borradla junto con su test (y sus líneas del `CMakeLists.txt`).

| Prueba mínima recomendada | Modelo en la referencia |
|---|---|
| El constructor guarda bien los datos | `HabitacionTest.ConstructorGuardaLosDatos` |
| Un dato no válido lanza una excepción (valor límite) | `HabitacionTest.PrecioCeroLanzaExcepcion` |
| Un método cambia el estado del objeto | `HabitacionTest.ReservarCambiaElEstado` |
| Si hay un `if` con un límite: los dos caminos y el límite | `SuiteTest.*` |
| Si hay herencia: el polimorfismo desde un puntero a la base | `SuiteTest.PolimorfismoDesdeUnPunteroALaBase` |
| La persistencia (obligatoria): guardar y cargar devuelve lo mismo | `AlmacenTest.GuardarYCargarDevuelveLoMismo` |

> **Con Claude**
>
> Pegad una de vuestras clases (el `.hpp` y el `.cpp`) y uno de vuestros tests, y pedidle: «*Propón 3 tests de GoogleTest más para esta clase, en el mismo estilo que este. Dime para cada uno si es de caja negra o de caja blanca y qué valor límite prueba*». **Antes de añadir un test, tenéis que saber explicar qué comprueba y por qué fallaría si la clase estuviera mal.**

## Resumen: la chuleta de la práctica

| Quiero... | Windows (PowerShell) | Mac (Terminal) |
|---|---|---|
| Configurar (1.ª vez) | `cmake -S . -B build -G Ninja` | `cmake -S . -B build` |
| Compilar (cada vez) | `cmake --build build` | `cmake --build build` |
| Ver cada test | `.\build\tests_gestor.exe` | `./build/tests_gestor` |
| Pasar los tests como el CI | `cd build`, `ctest --output-on-failure`, `cd ..` | igual |
| Pasar solo unos tests | `.\build\tests_gestor.exe --gtest_filter=Libro*` | `./build/tests_gestor --gtest_filter='Libro*'` |
| Subir el trabajo | `git add …`, `git commit -m "test(libro): …"`, `git push` | igual |
| Empezar de cero (borrar build) | `Remove-Item -Recurse -Force build` | `rm -rf build` |

| Tipo de prueba | Qué es | En la práctica |
|---|---|---|
| Unitaria | Una clase (o una función) sola | `test_libro`, `test_novedad`, `test_biblioteca` |
| Caja negra | Se escribe con lo que **promete** el `.hpp`, sin mirar el código. Casos normales y valores límite | `test_libro` |
| Caja blanca | Se escribe **mirando el código**: un test por camino del `if`, y el límite | `test_novedad` |
| Integración | Varias piezas **juntas** | `test_almacen` |
| Rendimiento | **Cuánto tarda** con muchos datos | `test_rendimiento` |

### Si algo falla

| Ves esto | Qué pasa | Qué hacer |
|---|---|---|
| `error: 'Habitacion' was not declared / has no member named …` | Queda algo del hotel sin adaptar | Ve a esa línea del test |
| `fatal error: suite.hpp: No such file or directory` | Falta cambiar el `#include` | `#include "novedad.hpp"` |
| Los tests nuevos no aparecen | El fichero no está en el `CMakeLists.txt`, o no lo has guardado | Paso 5, punto 4 |
| `redefinition of 'class LibroTest_…_Test'` | Dos tests con el mismo grupo y nombre (al copiar, se te olvidó cambiarlo) | Cambia el nombre de uno |
| El programa de tests se cierra de golpe (*Segmentation fault*, código de salida raro) | Has usado un puntero `nullptr` | `ASSERT_NE(p, nullptr);` antes de `p->` |
| He cambiado el código y no cambia nada | No has guardado | ● en la pestaña = sin guardar → `Ctrl+S` |

## Anexo · Instalar las herramientas (repaso de la Práctica 0)

Solo si en el paso 1.1 te falta alguna herramienta. Necesitas **conexión a internet** y unos **15-20 minutos**. Haz solo la parte de tu sistema y, dentro de ella, solo los pasos de lo que te falte. Al terminar, **cierra todas las terminales, abre una nueva** y repite las comprobaciones del paso 1.1.

| Herramienta | Windows | Mac |
|---|---|---|
| Compilador `g++` | MSYS2 → **W1-W3** | Xcode Command Line Tools → **M1** |
| `cmake` | MSYS2 → **W2-W3** | cmake.org → **M2** |
| `ninja` | MSYS2 → **W2-W3** | no hace falta |
| `git` | git-scm.com → **W4** | viene con las Command Line Tools → **M1** |
| VS Code | code.visualstudio.com → **W5** | code.visualstudio.com → **M3** |

### Windows

**W1. Instalar MSYS2** (el "gestor de paquetes" desde el que se instalan g++, CMake y Ninja):

1. Entra en **https://www.msys2.org** y pulsa el enlace del instalador, `msys2-x86_64-XXXXXXXX.exe` (la fecha del nombre cambia).
2. Ábrelo desde *Descargas* y pulsa **Siguiente** en todo, **sin cambiar la carpeta**: tiene que ser `C:\msys64`.
3. Al final, deja marcada **Run MSYS2 now** y pulsa **Finalizar**. Se abre una terminal **distinta** de PowerShell: la de MSYS2.

**W2. Instalar g++, CMake y Ninja.** En la terminal **MSYS2 MSYS** (tecla Windows → escribe MSYS2 → "MSYS2 MSYS"), primero actualiza:

Windows · terminal MSYS2 MSYS (no PowerShell)

```
$ pacman -Syu
```

Si pregunta `Proceed with installation? [Y/n]`, escribe `Y` y Enter. Si la ventana se cierra sola, es normal: ábrela otra vez y repite `pacman -Syu` hasta que diga `there is nothing to do`. Después instala las tres herramientas de una vez (confirma con `Y`):

Windows · terminal MSYS2 MSYS (no PowerShell)

```
$ pacman -S --needed mingw-w64-x86_64-gcc mingw-w64-x86_64-cmake mingw-w64-x86_64-ninja
```

**W3. Añadir la carpeta de MSYS2 al PATH** (para que PowerShell encuentre `g++`, `cmake` y `ninja`):

1. Tecla Windows → escribe **variables de entorno** → **Editar las variables de entorno del sistema** → botón **Variables de entorno…**
2. En la mitad de arriba (*Variables de usuario*), selecciona **Path** → **Editar…** → **Nuevo** → escribe exactamente `C:\msys64\mingw64\bin`
3. **Aceptar** en las tres ventanas.
4. **Cierra todas las ventanas de PowerShell** (y VS Code, si estaba abierto): solo las terminales nuevas ven el cambio.

**W4. Instalar Git:** entra en **https://git-scm.com/downloads/win**, pulsa **Click here to download**, abre el instalador y pulsa **Next** en todo (las opciones por defecto valen).

**W5. Instalar VS Code** (si no lo tienes): **https://code.visualstudio.com** → **Download for Windows**. Durante la instalación, deja marcada la casilla **Añadir a la RUTA (PATH)**.

> ✔ **Deberías ver**
>
> En una PowerShell **nueva**, `g++ --version`, `cmake --version`, `ninja --version` y `git --version` responden con un número de versión (como en el paso 1.1).

> ✘ **Si en vez de eso ves…**
>
> Siguen sin reconocerse: comprueba en W3 que la ruta está escrita **exactamente** como `C:\msys64\mingw64\bin` (sin espacios al final) y que has abierto una terminal nueva. Si aun así falla, reinicia el ordenador.
> En W2, `error: target not found`: hay una errata en el nombre de algún paquete. Copia el comando tal cual.

## Mac

**M1. Instalar g++ y Git (Xcode Command Line Tools).** En la Terminal:

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % xcode-select --install
```

Se abre una ventana: pulsa **Instalar**, acepta la licencia y espera (entre 5 y 15 minutos). Si en lugar de eso ves `command line tools are already installed`, ya los tienes: pasa a M2.

**M2. Instalar CMake:**

1. Entra en **https://cmake.org/download/**. En *Binary distributions*, descarga el fichero de macOS que termina en `macos-universal.dmg` (vale para chips Intel y Apple M).
2. Abre el `.dmg` y arrastra el icono de **CMake** a la carpeta **Applications**.
3. Abre **CMake** desde Aplicaciones (si macOS pregunta si quieres abrirla, pulsa **Abrir**).
4. **Este es el paso que más se olvida:** en el menú de arriba, **Tools → How to Install For Command Line Use**. Copia el comando que aparece, que es este:

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % sudo "/Applications/CMake.app/Contents/bin/cmake-gui" --install
Password:
```

Pégalo en la Terminal y pulsa Enter. Te pide la contraseña de tu Mac: escríbela (**no se ve nada mientras escribes**, es normal) y pulsa Enter. Cierra la Terminal, abre una nueva y comprueba con `cmake --version`.

> ✘ **Si en vez de eso ves…**
>
> `Failed to create symlink` o `No such file or directory` con `/usr/local/bin`: esa carpeta no existe en tu Mac. Créala con `sudo mkdir -p /usr/local/bin` y repite el comando `sudo "/Applications/CMake.app/…" --install`.
> Si usas **Homebrew**, también vale `brew install cmake` en lugar de los pasos 1-4.

**M3. Instalar VS Code** (si no lo tienes): **https://code.visualstudio.com** → **Download for Mac**, arrastra la aplicación a **Aplicaciones** y ábrela. Después, para poder usar `code .`: ⌘ Cmd + ⇧ Shift + P → **Shell Command: Install 'code' command in PATH**.

> ✔ **Deberías ver**
>
> En una Terminal **nueva**, `g++ --version`, `cmake --version` y `git --version` responden con un número de versión (como en el paso 1.1).